

(KCC2026 우수논문) DUET: 조기 종료 프록시 기반 캐시 구성을 활용한 비동기 추측 디코딩 기법

(DUET: Asynchronous Speculative Decoding with Early-exit Proxy-guided Cache Construction)

요약 비동기 speculative decoding은 target model이 검증하는 동안 다음 라운드의 speculation을 미리 준비해 두어 예측 성공 시에 drafting 지연을 숨길 수 있다. 다음 라운드의 입력은 검증 전에는 알 수 없으므로, draft model은 draft 분포만을 사용하여 검증 결과를 예측하는데, 검증 결과는 target 분포와의 관계로 결정되므로 예측 정확도가 떨어질 수 있는 문제가 있다. 본 논문에서는 target model이 검증하는 도중 중간 layer에서 얻은 early-exit proxy를 draft 모델에게 전달하여 예측 정확도를 높이는 DUET을 제안한다. DUET은 early-exit proxy로 각 검증 결과가 일어날 가능성을 추정하여, 가능성이 높은 결과에 준비 예산을 집중하고 그 speculation을 더 넓고 깊은 분기 구조로 확장한다. 이를 통해 더 높은 cache hit rate와 수락 길이를 얻는다.

키워드 : 대규모 언어 모델, 추측 디코딩, 비동기 추론, 조기 종료

Abstract Asynchronous speculative decoding prepares the next round's speculation while the target model verifies the current one, so that the drafting latency is hidden whenever the prediction succeeds. Since the next round's input is unknown before verification completes, the draft model predicts the verification outcome using only the draft distribution; because the outcome is governed by the relation between the draft and target distributions, this prediction can be inaccurate. This paper proposes DUET, which improves the prediction by forwarding to the draft model an early-exit proxy obtained from an intermediate layer while the target model is still verifying. Using this proxy, DUET estimates how likely each verification outcome is to occur, concentrates the preparation budget on the likely ones, and expands their speculation into a wider and deeper branching structure. As a result, DUET attains a higher cache hit rate and a longer acceptance length.

Key words : large language model, speculative decoding, asynchronous inference, early exit

1. 서론

대규모 언어 모델의 추론은 이전 토큰을 기반으로 다음 토큰을 생성하는 autoregressive 구조를 가지므로, 병렬 연산 성능이 높은 하드웨어에서도 토큰 단위의 순차 의존성이 디코딩 지연의 주요 병목으로 작용한다. Speculative decoding(SD)은 작은 draft model이 여러 draft token을 먼저 생성하고 큰 target model이 이를 한 번의 forward pass로 검증함으로써 이러한 병목을 완화한다 [1].

그러나 SD에서는 현재 라운드의 검증 결과가 다음 라운드의 입력을 결정하기 때문에, target model의 검증이 완료된 이후에야 다음 draft token 생성을 시작할 수 있다. 이를 개선하기 위해서, 비동기 SD는 target model이 현재 draft를 검증하는 동안 가능한 검증 결과에 대응하는 후속 speculation을 미리 생성하여 cache에 저장함으로써 이러한 순차 의존성을 완화한다. 실제 검증 결과와 일치하는

speculation이 cache에 존재하면 다음 라운드의 draft 생성 시간을 숨길 수 있으므로, 비동기 SD의 성능은 cache hit rate에 좌우된다 [2].

기존 비동기 SD 기법들은 draft 정보 혹은 target의 early-exit 정보 중 한쪽만을 활용하여 target의 검증 결과를 예측한다. 그러나 실제 검증 결과는 target과 draft의 출력 분포 간 관계로 결정되기에, 한쪽 정보만으로는 검증 결과를 정확하게 예측하기 어렵다.

본 논문에서는 이러한 한계를 해결하기 위해 DUET을 제안한다. DUET은 target model의 early-exit 정보를 draft 정보와 함께 사용하여 검증 결과를 사전에 예측하고, 실제 검증 결과가 될 가능성이 높은 reject 위치와 후속 token의 조합에 제한된 speculation 예산을 집중한다. 또한 선택된 각 후보의 speculation을 분기 구조로 확장하여, cache hit 시 더 긴 수락 길이를 확보한다. 이를 통해 DUET은 제한된 예산에서도 더 높은 cache hit rate와 수락 길이를 얻어, 결과적으로 높은 디코딩 처리량을 달성한다.

2. 배경

2.1. Speculative Decoding

Speculative decoding(SD)에서 draft model 은 현재까지 확정된 토큰 시퀀스인 prefix로부터 길이 L 의 draft token $x_{1:L}$ 을 생성하고, target model 은 이를 한 번의 forward pass로 검증한다 [1]. 각 위치 i ($1 \leq i \leq L$)의 draft token x_i 는 target 분포 p_T 와 draft 분포 p_D 에 따라 accept 된다. 식 (1)의 α_i 는 accept 확률이고, h_i 는 이전 token이 모두 accept된 뒤 위치 i 에서 처음으로 reject가 발생할 확률이다.

$$\alpha_i = \min\left(1, \frac{p_T(x_i)}{p_D(x_i)}\right), \quad h_i = \left(\prod_{j<i} \alpha_j\right)(1 - \alpha_i) \quad (1)$$

모든 draft token이 accept되면 다음 위치의 target 분포에서 bonus token을 생성한다. 위치 i 에서 reject가 발생하면 이후 draft token은 폐기되고, correction token v^* 는 식 (2)의 residual distribution $r_i(v)$ 에서 샘플링된다.

$$r_i(v) \propto \max(p_T(v) - p_D(v), 0), \quad v^* \sim r_i(v) \quad (2)$$

따라서 한 라운드의 검증 결과는 reject가 발생한 위치와 그 뒤에 이어지는 correction 또는 bonus token으로 결정되며, accept된 token 수에 이어지는 token 하나를 더한 값이 그 라운드에서 확정되는 수락 길이가 된다.

수락 길이를 늘리기 위해 SpecInfer [3]는 draft token을 분기 구조로 배치하여 여러 경로를 한 번의 forward pass로 함께 검증하며, EAGLE-2 [4]는 이러한 분기 구조를 draft 확률에 따라 문맥마다 다시 구성한다. 한편 LayerSkip [5]은 target model의 일부 layer만 통과시켜 draft token을 생성하는 self-speculative decoding을 제안하였다. 그러나 이러한 기법에서도 draft 생성과 검증 사이의 의존성은 그대로 남는다.

2.2. 비동기 Speculative Decoding

비동기 SD는 이 의존성을 제거하여, target model이 현재 draft token을 검증하는 동안 draft model이 다음 라운드의 speculation을 미리 준비한다. 다만 다음 라운드의 prefix는 이번 검증이 끝나야 확정되므로, draft model은 가능한 검증 결과 중 일부를 상정하여 각각에 대한 speculation을 미리 생성하고 speculation cache에 저장한다. 이렇게 상정한 개별 결과를 검증 결과 후보라 하며, 각

후보는 reject가 발생하는 위치 i 와 그 뒤에 이어지는 token v 의 쌍 (i, v) 로 나타낸다. 실제 검증 결과와 일치하는 후보가 cache에 있으면(cache hit) 저장된 speculation을 즉시 반환하고, 없으면(cache miss) draft 생성 시간이 그대로 노출된다.

따라서 비동기 SD의 성능은 cache hit rate에 좌우되며, 이는 어느 위치에 어떤 token을 후보로 두는지에 따라 결정된다. 가능한 후보는 위치 i 와 token v 의 모든 조합만큼 존재하는 반면, 검증 시간 안에 생성할 수 있는 speculation은 제한되며 그중 실제로 쓰이는 것은 cache hit된 하나뿐이다.

Saguaro [2]는 앞쪽 위치일수록 reject가 발생할 확률이 높다는 점에서 더 많은 후보예산을 배분하고, draft 분포의 상위 token으로 후보를 선택한다. Mirror-SD [6]는 위치별로 균등하게 후보예산을 배분하며, target의 early-exit 분포에서 상위 token을 후보로 선택한다.

그러나 식 (1), (2)에서 보듯 검증 결과는 target 분포와 draft 분포의 관계로 결정된다. accept 확률은 두 분포의 비로, correction token은 두 분포의 차이로 정해지므로, 어느 한쪽만으로는 어느 위치에서 reject가 발생하고 그 뒤에 어떤 token이 올지를 문맥에 따라 예측하기 어렵다.

3. 동기

본 절에서는 target model의 early-exit proxy 정보가 비동기 SD의 cache 구성에 활용될 수 있는지 분석한다. 여기서 early-exit proxy는 검증 forward가 최종 layer에 도달하기 전, 중간 layer의 hidden state로부터 얻은 분포를 의미한다. 측정에는 LayerSkip [5] 방식으로 학습된 target model을 사용하였다.

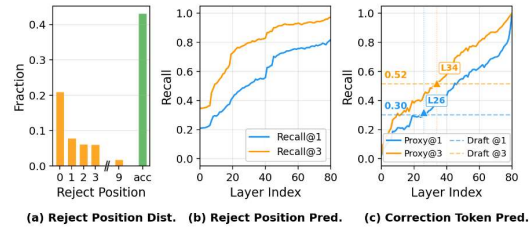


그림 1. Early-exit proxy 기반 검증 결과 예측
Fig. 1. Verification outcome prediction with an early-exit proxy

그림 1(a)는 실제 검증 결과의 분포이다. 모든 draft token 이 accept 되는 경우가 가장 많고 reject 는 초반 위치에 집중되는데, 이는 식 (1)의 h_i 가 이전 위치의 accept 확률을 누적하기 때문이다. 따라서 모든 위치에 후보를 균등하게 배분하면 거의 발생하지 않는 결과에 후보 예산이 소모된다.

그림 1(b)와 (c)는 식 (1)과 (2)에서 target 분포를 proxy 분포로 대체하여 계산했을 때, 상위 후보가 실제 검증 결과의 reject 위치와 correction token 을 각각 포함하는 비율을 early-exit layer 위치에 따라 보여주고 있다. Proxy 를 뒤쪽 layer 에서 추출할수록, proxy 를 이용한 경우가 draft 분포만 사용하는 경우보다 correction token 예측 정확도가 높다.

즉 early-exit proxy 는 검증이 끝나기 전에 검증 결과와 관련 있는 정보를 제공하므로, DUET 은 이를 후보 선정에 활용한다. 다만 proxy 를 늦게 추출할수록 후보 예측은 정확해지는 반면 draft 가 이를 활용할 시간은 줄어든다. 따라서 exit layer 는 이상층 관계를 고려하여 정해야 하며, 본 논문에서는 예비 실험에서 가장 높은 처리량을 보인 56 번째 layer 를 사용한다. 이 경우 남은 layer 의 검증 시간 동안 draft 는 네 번의 forward 를 수행할 수 있다.

4. DUET

4.1. 전체 구조

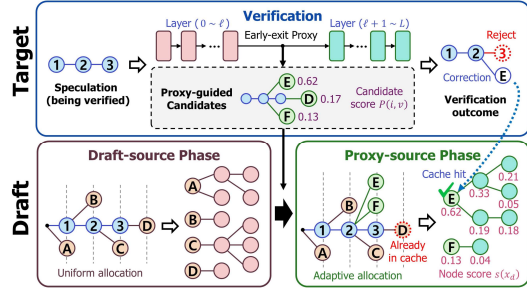


그림 2. DUET 의 전체 구조

Fig. 2. Overall architecture of DUET

그림 2는 DUET 의 한 라운드를 보여준다. 위쪽은 target 이, 아래쪽은 draft 가 동일 시간 동안 수행하는 일이다. target 은 draft 가 이전 라운드에 준비해 둔 speculation 을 검증한다. 이 검증 forward 가 early-exit layer 를 통과할 때 나오는 proxy 정보를 통해, 각 검증 결과 후보가 실제 결과가 될 가능성을 점수로 계산하고 상위 후보를 점수와 함께 draft 로 보낸다.

draft 는 이 점수가 도착하기 전과 후, 두 phase 로 나누어 speculation cache 를 구성한다. 각각의 phase 에서는 검증 결과 후보를 정하는 일(4.2 절)과 그 후보에서부터 이어지는 speculation 을 생성하는 일(4.3 절)이 차례로 수행된다.

Target 의 검증이 끝나면 확정된 결과로 draft 가 미리 준비한 cache 를 조회한다. 그림 2 의 예제에서는 검증 결과, draft token 3이 reject 되고 식 (2)에 의해 correction token E 가 샘플링되었다. 이 결과가 draft 가 준비한 후보 E 와 일치하므로 cache hit 이 발생하고 후보 E 에 대한 speculation 이 즉시 반환되어 다음 라운드가 시작된다.

4.2. Dual-source Cache 구성

DUET 은 cache 를 구성할 때 두 정보원을 순서대로 사용한다. 첫 번째는 draft-source(DS) phase 이다. 이 시점에는 target 쪽 정보가 없으므로 draft 분포만으로 후보를 정하며, 가능한 위치마다 같은 수의 후보를 배분한다. 그림에서 A, B, C 는 각각 draft token 1, 2, 3 에서 reject 되는 경우이고 D 는 모든 draft token 이 accept 되는 경우로, 위치마다 하나씩 고르게 놓여 있다. 각 위치에 할당되는 후보 토큰(A, B, C, D)은 검증 중인 draft token(1, 2, 3)을 제외한 draft 분포의 상위 token 으로 정한다. DS phase 는 target 의 proxy 정보를 기다리지 않으므로 기존 비동기 방식의 병렬성을 유지하면서, 모든 위치에 최소한의 speculation 을 확보한다.

두 번째는 proxy-source(PS) phase 로, target 이 proxy 정보를 통해 선정한 후보를 전송받아 cache 를 보완한다. 각 후보 (i, v) 의 점수는 식 (3)과 같이 정의된다.

$$P(i, v) = \hat{h}_i \cdot \hat{r}_i(v) \quad (3)$$

여기서 $\hat{h}_i$ 는 위치 i 에서 reject 가 발생할 확률이고, $\hat{r}_i(v)$ 는 그 위치에서 token v 가 correction token 으로 샘플링될 확률이다. 두 값은 식 (1)의 h_i 와 식 (2)의 $r_i(v)$ 에서 검증 전에는 알 수 없는 target 분포 p_r 를 proxy 분포 p_E 로 대체하여 얻는다. 모든 draft token 이 accept 되는 경우도 같은 점수 식으로 다루는데, $\hat{h}_i$ 는 모든 draft token 이 accept 될 확률이 되고, $\hat{r}_i$ 는 proxy 분포 p_E 가 된다.

PS phase 에서는 점수가 높은 상위 N 개를 후보로 선택한다. 위치별 개수를 미리 정하지 않으므로,

후보는 reject 가 발생할 가능성이 높은 위치와 그 위치에서 이어질 가능성이 높은 token 에 집중된다. 그림에서 점수가 높은 후보는 $E(0.62)$, $D(0.17)$, $F(0.13)$ 이다. draft 는 이 가운데 DS phase 가 이미 만든 D 를 제외하고 E 와 F 만 cache 에 추가한다. E 와 F 는 모두 세 번째 위치에 놓여, 위치마다 같은 수를 배분한 DS phase 를 보완한다. 결과적으로 DUET 은 실제 검증 결과가 될 가능성이 높은 후보에 예산을 집중하여 더 높은 cache hit rate 를 달성할 수 있다.

4.3. 다중 검증 결과를 위한 분기 구성

DUET 은 speculation 을 단일 경로가 아닌 분기 구조로 준비하여 수락 길이를 늘린다. 다만 기존 SD가 확정된 검증 결과 하나에서 분기하는 것과 달리, 비동기 SD는 4.2절에서 선정한 여러 후보에서 동시에 speculation 노드를 생성해야 한다. 이때 모든 노드를 같은 폭으로 분기시키면 draft 비용이 크게 증가하는 반면, 이 중 실제로 쓰이는 것은 hit 된 하나뿐이다. 따라서 아직 확정되지 않은 후보들 사이에서 제한된 노드 생성 예산을 어떻게 나눌 것인지가 문제가 된다.

각 phase에서 draft는 K 번 forward를 수행하며, 매 forward마다 W 개의 노드를 확장한다. 따라서 생성할 수 있는 노드는 최대 $W \times K$ 개이며, 이것이 노드 예산이 된다. DUET 은 모든 후보의 노드에 점수를 매기고, 이 예산을 점수에 따라 배분한다. 후보 (i, v) 아래 깊이 d 에 있는 노드 x_d 의 점수는 식 (4)와 같이 정의된다.

$$s(x_d) = P(i, v) \cdot \prod_{j=1}^d p_D(x_j | (i, v), x_{1:j-1}) \quad (4)$$

여기서 $x_1, \dots, x_{d-1}$ 은 후보로부터 x_d 에 이르는 경로이고, $P(i, v)$ 는 식 (3)의 후보 점수이다. 즉 노드 점수 $s(x_d)$ 는 그 후보가 실제 검증 결과가 될 확률과 draft 분포에서 그 경로가 갖는 확률의 곱을 나타낸다. 단 DS phase의 후보에는 proxy 정보가 없으므로, 위치 i 까지 draft token이 이어질 확률과 token v 의 draft 확률의 곱을 $P(i, v)$ 대신 사용한다.

draft는 매 forward마다 모든 후보의 말단 노드를 점수 순으로 비교하여 상위 W 개만 확장한다. 점수가 높은 후보는 자식 노드의 점수도 높게 이어지므로 깊고 넓은 분기 구조를 이루고, 점수가 낮은 후보는 얇은 경로로 형성된다. 예를 들어 그림 2에서 점수가 0.62인 후보 E 아래에는 다섯 개의 노드가 배정된 반면, 0.13인 F 아래에는 하나만 배정된다. 이처럼

노드 예산은 사전에 정해지지 않고 경쟁의 결과로 결정되며, 실제 검증 결과가 될 가능성이 높은 후보에 집중된다. 그 후보가 cache hit될 때 더 넓고 깊은 분기 구조가 함께 검증되므로, 수락 길이가 늘어난다.

4.4. 통신 및 동기화 최적화

후보 선정은 target의 검증 도중 수행된다. 따라서 LM head 계산과 GPU 간 통신, 후보 선별과 전송을 기본 stream에서 차례로 실행하면 이 시간이 모두 검증 지연에 그대로 더해진다. DUET은 전송량을 줄이고 이들 연산을 검증과 겹쳐 수행하여 그 영향을 최소화한다.

먼저 전송량을 줄인다. exit layer의 hidden state에 LM head를 적용하면 검증 위치마다 어휘 크기의 logit이 만들어지는데, 후보 선정을 draft에 맡기려면 이를 모두 보내야 한다. 검증 위치가 10개이고 어휘 크기가 32,000이면 전송량은 약 640KB이다. DUET은 target에서 후보 점수까지 계산한 뒤 상위 N 개의 후보만 전송하며, 각 후보의 점수는 token 번호가 쓰지 않는 상위 비트에 실어 함께 전달한다. N 이 32이면 전송량은 512 바이트로 줄어든다.

다음으로 GPU 간 대기를 줄인다. 어휘 전체의 logit은 여러 GPU에 나뉘어 계산되므로 이를 한 GPU로 모아야 하며, 그동안 나머지 GPU는 대기한다. DUET은 한 GPU가 어휘 전체의 LM head를 보유하여 이 과정을 없애므로, 나머지 GPU는 exit layer에서 아무 작업 없이 다음 layer 연산을 바로 시작한다. LM head 연산을 하는 GPU도 점수 계산 및 전송을 별도의 CUDA stream에 두므로, 기본 stream은 이를 기다리지 않고 다음 layer 연산을 이어간다. 전송 또한 비차단으로 처리하여 target은 draft의 수신을 기다리지 않는다.

5. 실험 및 평가

5.1. 실험 환경

성능 평가는 LayerSkip 방식으로 학습된 Llama2-70B를 target model로, TinyLlama-1.1B-Chat-v1.0을 draft model로 사용하여 수행하였다 [5]. 실험은 NVIDIA RTX PRO 6000 Blackwell GPU 3개를 이용하여 진행하였으며, target model은 2개의 GPU에 tensor parallelism으로 분할하고 비동기 draft 실행은 나머지 1개에 할당하였다.

데이터셋은 다중 턴 대화, 번역, 요약, 질의응답, 수학 추론, 검색 증강 생성의 6개 서브태스크로

구성된 Spec-Bench 를 사용하였다 [7]. 생성 조건은 temperature 0.7, 최대 출력 token 수 1,024 로 고정하였으며, 모든 측정은 batch 크기 1 에서 수행하였다. DUET 은 exit layer 를 56 번째 layer 로 두고, DS phase 와 PS phase 에서 각각 8 번과 4 번의 draft forward 를 수행하도록 설정하였으며, 비교 기법들의 설정은 예비 실험에서 가장 높은 처리량을 보인 값을 사용하였다.

5.2. 전체 성능 비교

DUET 은 AR 대비 3.57 배의 처리량을 달성하여 비교 기법 중 가장 높은 성능을 보였다. 같은 비동기 방식인 Saguaro 와 비교하면 cache hit rate 가 33% 상승하고 수락 길이가 12% 증가하여 처리량이 10.4% 향상되었다. hit rate 의 상승은 준비할 후보를 draft 분포뿐만 아니라 target 의 early-exit proxy 정보로 보완한 결과이며, 수락 길이의 증가는 각 후보의 speculation 을 분기 구조로 확장한 결과이다. 한편 동기식 분기 구조 기법인 SpecInfer 는 수락 길이가 4.97 로 가장 길지만, 검증하는 노드 수가 많고 draft 생성 시간도 그대로 노출되어 DUET 보다 낮은 처리량을 보였다.

표 1. 디코딩 기법별 성능 비교

Table 1. Performance comparison of decoding methods

Method	Decode TPS	AL	Hit rate
AR (baseline)	19.1 ($\times 1.00$)	-	-
EAGLE-2	51.3 ($\times 2.69$)	3.77	-
Vanilla SD	53.5 ($\times 2.80$)	4.26	-
SpecInfer	57.8 ($\times 3.03$)	4.97	-
Saguaro	61.7 ($\times 3.23$)	4.18	0.603
DUET (Ours)	68.1 ($\times 3.57$)	4.68	0.804

5.3. Dual-source 구성의 효과

검증 위치당 평균 후보 수, 즉 후보 예산을 변화시키며 hit rate 를 측정하였다. Uniform 은 모든 위치에 같은 수의 후보를 배분하고, Geometric 은 뒤쪽 위치일수록 그 결과가 발생할 확률이 낮다는 점에서 앞쪽에 더 많은 후보를 배분한다 [2]. 두 방식 모두 각 위치에서 이어지는 token 은 draft 분포의 상위 token 에서 고른다. Only-Proxy 는 반대로 proxy

정보만으로 후보를 정하여 식 (3)의 점수 상위 후보로 cache 를 채운다. DUET 은 draft 와 proxy 두 정보원을 함께 사용하며, 후보 예산을 DS phase 와 PS phase 에 절반씩 나눈다.

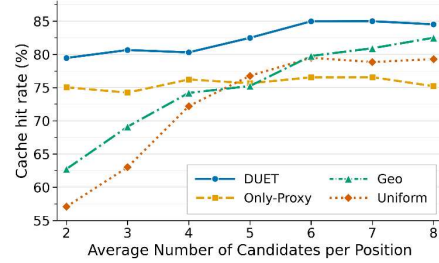


그림 3. 위치당 평균 후보 수에 따른 hit rate

Fig. 3. Hit rate versus the average number of candidates per position

Uniform 과 Geometric 은 후보 예산이 작을 때 hit rate 가 낮지만 예산에 따라 꾸준히 상승한다. 반면 Only-Proxy 는 예산 2 에서 이미 75%를 넘어 두 방식보다 크게 높으면서도, 예산을 늘려도 크게 달라지지 않는다. 이는 proxy 점수가 후보를 고르는 방식에서 비롯된다. 식 (2)의 residual 을 근사한 $\hat{r}_i(v)$ 는 $p_E(v) > p_D(v)$ 인 token 에서만 0 보다 큰 값을 가지며, 반대의 경우에는 해당 token 이 실제 correction token 이 될 수 있음에도 점수가 0 이 되어, 예산과 무관하게 후보에서 제외된다.

반면 DUET 의 DS phase 는 $p_D(v)$ 가 큰 영역을 우선 포함하는데 이 영역이 바로 proxy 의 점수가 0 이 되기 쉬운 곳이므로, PS phase 가 제외하는 후보를 DS phase 가 보완할 수 있다. DUET 은 예산을 두 phase 에 나누어 이러한 상보성을 활용하며, 모든 구간에서 가장 높은 hit rate 를 유지한다.

5.4. 수락 길이 분석

그림 4 는 DS 와 PS 가 준비한 speculation 이 적중한 경우와 cache miss 가 발생한 경우에 대해, 수락 길이가 특정 값 이상일 확률을 나타낸다.

DS 가 적중한 경우는 실제 검증 결과가 draft 분포의 상위 token 에 포함된 경우이고, PS 가 적중한 경우는 그렇지 않아 target 정보로 보완된 경우이며, cache miss 는 두 정보원 모두 준비하지 못한 경우이다. DS hit, PS hit, cache miss 의 순서로 곡선이 낮아지며 이 순서는 모든 구간에서 유지된다. 즉

draft가 이번 검증 결과를 맞히지 못한 문맥일수록 그 이후의 경로도 draft가 생성하기 어렵다고 볼 수 있다.

이때 PS 폭선은 더 가파르게 감소하여 같은 forward를 추가하더라도 수락 길이로 환산되는 이득이 작다. DUET이 PS phase에 더 적은 forward를 배정하는 것은 이러한 특성을 반영한 것이며, 그만큼 exit layer를 뒤로 미루어 proxy 자체의 품질을 높일 수 있다.

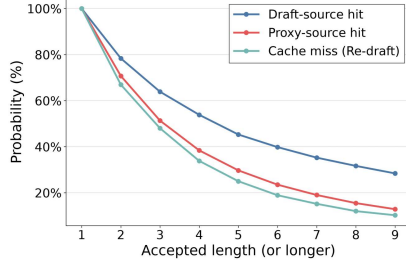


그림 4. cache 조회 결과별 수락 길이 분포

Fig. 4. Acceptance length distribution by cache lookup outcome

표 2. 단일 경로와 분기 구조의 비교

Table 2. Comparison of single and branching structures

Config	Decode TPS	AL	DS AL	PS AL	Step (ms)
Chain	66.4	4.33	5.10	2.99	66.1
Tree	68.1	4.68	5.54	3.27	69.6

표 2는 단일 경로와 분기 구조 방식을 비교하는데, 분기 구조는 같은 draft 비용 안에서 수락 길이를 늘렸으며, 두 phase 모두에서 비슷한 폭으로 개선되었다. 검증하는 노드 수가 늘어 스텝당 지연도 함께 증가하지만, 수락 길이의 증가가 이를 상회하여 처리량이 향상되었다.

5.5. 지연 시간 분해

그림 5는 한 디코딩 스텝의 시간을 target과 draft로 나누어 각 구간의 비율로 나타낸 것이다. context align은 직전 검증 결과를 draft의 KV cache에 반영하는 구간이다.

cache hit인 경우(그림 5(a)) DUET이 target에 추가하는 exit layer의 proxy 계산은 스텝의 1.7%에 그친다. 이는 GPU 간 통신 없이 LM head 및 proxy 계산을 수행하고, 이를 별도의 CUDA stream에 두어

뒤따르는 layer의 검증과 겹치기 때문이다. draft측의 작업은 target의 검증이 진행되는 구간 안에 모두 들어가므로, cache를 구성하는 데 드는 시간은 지연으로 드러나지 않는다.

반면 cache miss인 경우(그림 5(b)) draft가 확정된 검증 결과로부터 speculation을 새로 생성해야 하며, 이 시간이 그대로 target의 stall로 나타난다. 이 구간은 한 디코딩 스텝의 13%이며, miss 스텝은 hit 스텝보다 약 15% 길어진다.

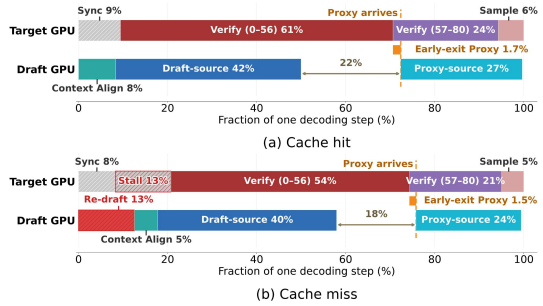


그림 5. cache hit과 miss 시의 디코딩 스텝 시간 분해

Fig. 5. Decoding step time breakdown under cache hit and cache miss

6. 결론

본 논문에서는 비동기 SD의 cache hit rate와 수락 길이를 함께 높이는 DUET을 제안하였다. DUET은 target model이 검증하는 도중 중간 layer에서 얻은 early-exit proxy를 두 가지에 활용한다. 첫째, 각 검증 결과 후보가 실제 결과가 될 가능성을 점수로 추정하고 점수가 높은 후보를 cache에 추가하여 cache hit rate를 높인다. 둘째, 각 후보의 speculation을 확장할 때 제한된 노드 예산을 가능성이 높은 후보에 집중하여 수락 길이를 늘린다. 실험을 통해 DUET이 기존 비동기 SD보다 높은 처리량을 달성함을 확인하였다. 다만 DUET은 중간 layer의 출력이 최종 layer의 분포와 유사할 것을 전제하므로 현재는 LayerSkip 방식으로 학습된 target model에 적용되며, 재학습 없이 이러한 중간 출력을 얻는 방법을 향후 과제로 남긴다.

참 고 문 헌

- [1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," Proc. of the 40th International Conference on Machine Learning (ICML), pp. 19274-19286, 2023.
- [2] T. Kumar, T. Dao, and A. May, "Speculative Speculative Decoding," Proc. of the 14th International Conference on Learning Representations (ICLR), 2026.
- [3] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, "SpecInfer: Accelerating Large Language Model Serving with Tree-based Speculative Inference and Verification," Proc. of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 3, pp. 932-949, 2024.
- [4] Y. Li, F. Wei, C. Zhang, and H. Zhang, "EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees," Proc. of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 7421-7432, 2024.
- [5] M. Elhoushi, A. Shrivastava, D. Liskovich, B. Hosmer, B. Wasti, L. Lai, A. Mahmoud, B. Acun, S. Agarwal, A. Roman, A. Aly, B. Chen, and C.-J. Wu, "LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding," Proc. of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pp. 12622-12642, 2024.
- [6] N. Bhendawade, K. Nishu, A. Kundu, C. Bartels, M. Cho, and I. Belousova, "Mirror Speculative Decoding: Breaking the Serial Barrier in LLM Inference," arXiv preprint arXiv:2510.13161, 2025.
- [7] H. Xia, Z. Yang, Q. Dong, P. Wang, Y. Li, T. Ge, T. Liu, W. Li, and Z. Sui, "Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding," Findings of the Association for Computational Linguistics: ACL 2024, pp. 7655-7671, 2024.